

SponsorX Implementation Guide

The stack, wired. Pinned versions, repo layout, the Prisma schema, and the six patterns that are load-bearing — tenant scoping, transactional job enqueue, the tracking funnel, Zoho loop prevention, signature capture and the authorization test matrix.

Runtime Node 22 · TypeScript 5 **Verified** npm registry, Sept 2026 **Scope** Phase 1 loop

01 Pin these versions

Checked against npm today

PACKAGE	VERSION	MIGRATION NOTE
next	16.3.4	App Router. <code>params</code> is a Promise — <code>const { code } = await params</code> .
react	19.2.8	Server Components default; <code>useActionState</code> for form mutations.
prisma / @prisma/client	7.10.0	Not 8.x — that is still release-candidate. v7 uses <code>prisma.config.ts</code> and a generated-client output path you must set explicitly.
zod	4.5.4	v4 changed error customisation and <code>.default()</code> inference. Do not copy v3 snippets.
tailwindcss	4.3.3	CSS-first config. There is no <code>tailwind.config.js</code> — theme lives in <code>@theme</code> in your CSS.
@clerk/nextjs	7.9.1	Organizations map to tenants; custom session claims carry the role.
pg-boss	12.30.0	Queues live in your Postgres. Run migrations on worker boot, not in the web app.
react-hook-form	7.87.0	With <code>@hookform/resolvers</code> for the Zod bridge.
recharts	3.10.1	Client components only — wrap in <code>"use client"</code> islands.
playwright	1.63.0	Two jobs: E2E tests, and PDF rendering on the worker.
resend / twilio	6.26.0 / 6.1.0	Twilio v6 is ESM-first.
vitest	5.0.0	Unit and the authorization matrix in \$09.

Pin exactly, no carets, for the first two sprints. A pod of three across time zones does not need a transitive minor bump breaking the build on a Tuesday. Move to ranges once CI is trustworthy.

02 Repo layout

One repository, two deployables: the Next app and the worker. Both import from `src/server`, so a job and a route handler can never disagree about what a Campaign Order is.

```
sponsorx/
├─ prisma/
│  └─ schema.prisma
│    └─ migrations/
├─ prisma.config.ts
├─ src/
│  └─ app/
│     └─ (public)/
│        └─ page.tsx
│           └─ packages/page.tsx
│              └─ join/page.tsx
│                 └─ athletes/[slug]/page.tsx
│                    └─ r/[token]/page.tsx
│                       └─ t/[code]/route.ts
│                          └─ (app)/
│                             └─ sponsor/... athlete/... property/... admin/...
│                                └─ api/v1/
│                                   └─ openapi.json/route.ts
│                                      └─ athletes/route.ts
│                                         └─ campaign-orders/[id]/accept/route.ts
│                                            └─ metrics/events/route.ts
│                                               └─ webhooks/zoho/route.ts
├─ server/
│  └─ db.ts
│     └─ actor.ts
│        └─ scope.ts
│           └─ outbox.ts
│              └─ audit.ts
│                 └─ domain/
│                    └─ campaign.ts, reward.ts, earnings.ts ...
├─ contracts/
│  └─ athlete.ts campaign.ts reward.ts
│     └─ registry.ts
│        └─ components/ui/
│           └─ generated/prisma/
├─ worker/
│  └─ index.ts
│     └─ jobs/
│        └─ zoho-push.ts notify.ts rollup-metrics.ts render-report.ts
│           └─ expire-holds.ts
├─ tests/
│  └─ authz.matrix.test.ts
│     └─ e2e/campaign-loop.spec.ts
```

WHY NOT A MONOREPO

Turborepo earns its keep when you have separate deploy artifacts with different dependency trees. Here the worker imports the same Prisma client and the same domain functions as the web app. Two `package.json` files would mean two lockfiles drifting apart on the code that decides how much an athlete gets paid.

03 The schema

Core tables; §20 lists the rest

Four conventions run through every model: `tenantId` on everything, state enums lifted verbatim from §21, external IDs plus a sync-origin marker on anything Zoho touches, and events stored as rows rather than counters.

```

// prisma/schema.prisma
generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db { provider = "postgresql"; url = env("DATABASE_URL") }

enum Role { SUPER_ADMIN BTG_ADMIN SALES CAMPAIGN_MGR NETWORK_MGR FINANCE
            ATHLETE GUARDIAN PROPERTY_MGR SPONSOR_ADMIN SPONSOR_ANALYST SERVICE }

enum AthleteState { DRAFT SUBMITTED UNDER_REVIEW APPROVED CHANGES_REQUESTED
                   REJECTED ACTIVE SUSPENDED }
enum CampaignState { DRAFT STAFFING APPROVAL ACTIVE REPORTING COMPLETED CANCELLED }
enum OrderState { DRAFT SENT ACCEPTED REJECTED ACTIVE COMPLETED CANCELLED }
enum DeliverableState { NOT_STARTED DRAFT_SUBMITTED BTG_REVIEW SPONSOR_REVIEW
                      APPROVED PUBLISHED VERIFIED }
enum EarningState { PENDING ELIGIBLE APPROVED_FOR_PAYOUT PAID HELD DISPUTED }
enum RewardState { DRAFT ACTIVE PAUSED EXPIRED ARCHIVED }
enum EventType { SCAN LANDING CLAIM REDEEM }
enum MetricSource { VERIFIED_API VERIFIED_MANUAL SELF_REPORTED ESTIMATED ATTRIBUTED }
enum SyncOrigin { SPONSORX ZOHO }

model Tenant {
  id      String  @id @default(cuid())
  name    String
  createdAt DateTime @default(now())
}

model User {
  id      String  @id @default(cuid())
  tenantId String
  clerkId String  @unique           // identity lives in Clerk; this row is the mirror
  email   String
  roles   Role[]
  athleteId String? @unique
  sponsorId String?
  athlete Athlete? @relation(fields: [athleteId], references: [id])
  @@index([tenantId])
}

model Athlete {
  id      String  @id @default(cuid())
  tenantId String
  slug     String  @unique           // public profile URL
  legalName String
  displayName String
  birthDate DateTime?               // drives the guardian path
  city     String?
  state    String?
  sport    String
  school   String?
  gradYear Int?
  state_   AthleteState @default(DRAFT) @map("state")
  tier     String?
  guardianId String?
  // Zoho
  zohoId   String? @unique
  lastSyncOrigin SyncOrigin?
  lastSyncHash String?
  createdAt DateTime @default(now())
  socials   AthleteSocial[]
  rates     AthleteRate[]
}

```

```

    orders          CampaignOrder[]
    user             User?
    @@index([tenantId, state_])
}

model AthleteSocial {
  id                String          @id @default(cuid())
  athleteId         String
  platform          String
  handle            String
  followers          Int?
  avgViews           Int?
  source            MetricSource @default(SELF_REPORTED)
  capturedAt         DateTime       @default(now())
  athlete           Athlete         @relation(fields: [athleteId], references: [id])
  @@unique([athleteId, platform])
}

model NilJob {
  id                String @id
  tenantId          String
  name              String
  baseLow           Int
  baseHigh          Int
  sellLow           Int
  sellHigh          Int
}

model AthleteRate {
  id                String @id @default(cuid())
  athleteId         String
  jobId             String
  amount            Int
  version           Int @default(1)
  athlete           Athlete @relation(fields: [athleteId], references: [id])
  @@unique([athleteId, jobId, version])
}

model Sponsor {
  id                String          @id @default(cuid())
  tenantId          String
  name              String
  zohoAccountId     String?         @unique
  lastSyncOrigin    SyncOrigin?
  lastSyncHash      String?
  campaigns         Campaign[]
}

model Campaign {
  id                String          @id @default(cuid())
  tenantId          String
  sponsorId         String
  name              String
  budget            Int
  startDate         DateTime
  endDate           DateTime
  state             CampaignState @default(DRAFT)
  zohoDealId        String?
  sponsor           Sponsor         @relation(fields: [sponsorId], references: [id])
  orders            CampaignOrder[]
  rewards           Reward[]
  @@index([tenantId, state])
}

```

```

}

model CampaignOrder {
    id            String            @id @default(cuid())
    tenantId      String
    campaignId    String
    athleteId     String
    jobId         String
    compensation   Int
    usageRights   String
    exclusivity   String?
    dueDate       DateTime
    state         OrderState       @default(DRAFT)
    acceptedAt    DateTime?
    acceptanceId  String?          @unique           // → AgreementAcceptance
    campaign      Campaign         @relation(fields: [campaignId], references: [id])
    athlete       Athlete          @relation(fields: [athleteId], references: [id])
    deliverables  Deliverable[]
    earning        Earning?
    @@unique([campaignId, athleteId, jobId])
    @@index([tenantId, state])
}

model Deliverable {
    id            String            @id @default(cuid())
    tenantId      String
    orderId       String
    title         String
    dueDate       DateTime
    state         DeliverableState @default(NOT_STARTED)
    publishedUrl  String?
    publishedAt   DateTime?
    order         CampaignOrder    @relation(fields: [orderId], references: [id])
    assets        CreativeAsset[]
    link          TrackingLink?
    metrics       MetricDaily[]
    @@index([tenantId, state, dueDate])
}

model CreativeAsset {
    id            String            @id @default(cuid())
    deliverableId String
    version       Int
    r2Key         String            // signed-URL access only
    uploadedBy    String
    uploadedAt    DateTime          @default(now())
    deliverable   Deliverable       @relation(fields: [deliverableId], references: [id])
    @@unique([deliverableId, version])
}

model MetricDaily {
    id            String            @id @default(cuid())
    deliverableId String
    day           DateTime          @db.Date
    views         Int              @default(0)
    engagements   Int              @default(0)
    source        MetricSource
    enteredBy     String?
    deliverable   Deliverable       @relation(fields: [deliverableId], references: [id])
    @@unique([deliverableId, day, source])
}

```

```

model TrackingLink {
  id          String      @id @default(cuid())
  tenantId    String
  code        String      @unique           // short, opaque
  deliverableId String    @unique
  destinationUrl String
  deliverable Deliverable @relation(fields: [deliverableId], references: [id])
}

model Reward {
  id          String      @id @default(cuid())
  tenantId    String
  campaignId  String
  offerText   String
  singleUse   Boolean     @default(true)
  expiresAt   DateTime
  terms       String
  state       RewardState @default(DRAFT)
  campaign    Campaign    @relation(fields: [campaignId], references: [id])
  tokens      RewardToken[]
}

model RewardToken {
  id          String      @id @default(cuid())
  rewardId    String
  athleteId   String?     // per-athlete attribution
  token       String      @unique           // opaque, never the record id
  qrKey       String?     // R2 object
  reward      Reward      @relation(fields: [rewardId], references: [id])
  events      RewardEvent[]
}

model RewardEvent {
  id          String      @id @default(cuid())
  tenantId    String
  tokenId     String
  type        EventType
  at          DateTime    @default(now())
  city        String?
  region      String?
  fanEmail    String?
  token       RewardToken @relation(fields: [tokenId], references: [id])
  @@index([tenantId, type, at])
}

model Earning {
  id          String      @id @default(cuid())
  tenantId    String
  athleteId   String
  orderId     String      @unique
  gross       Int
  adjustment  Int         @default(0)
  state       EarningState @default(PENDING)
  taxYear     Int
  paidAt      DateTime?
  reference   String?
  order       CampaignOrder @relation(fields: [orderId], references: [id])
  @@index([athleteId, taxYear])           // year-end reporting rollup
}

model Agreement {
  id          String @id @default(cuid())

```

```

kind      String          // COLLAB | CAMPAIGN_ORDER | GUARDIAN | RELEASE
version   Int
bodyHash  String          // sha256 of the exact text shown
effectiveAt DateTime
@@unique([kind, version])
}

model AgreementAcceptance {
  id        String    @id @default(cuid())
  tenantId  String
  agreementId String
  userId    String
  bodyHash  String    // copied at accept time
  ip        String
  userAgent String
  acceptedAt DateTime @default(now())
  guardianId String?
}

model OutboxJob {
  id        String    @id @default(cuid())
  tenantId  String
  name      String
  payload   Json
  createdAt DateTime @default(now())
  dispatchedAt DateTime?
  @@index([dispatchedAt, createdAt])
}

model AuditLog {
  id        String    @id @default(cuid())
  tenantId  String
  actorId   String?
  action    String
  entity    String
  entityId  String
  before    Json?
  after     Json?
  at        DateTime @default(now())
  @@index([tenantId, entity, entityId])
}

```

TWO INDEXES PRISMA WILL NOT CREATE FOR YOU

```

-- One redemption per single-use token, enforced by Postgres, not by application code.
CREATE UNIQUE INDEX reward_single_redeem
  ON "RewardEvent" ("tokenId")
  WHERE type = 'REDEEM';

-- Outbox drain: only ever scans undispached rows.
CREATE INDEX outbox_pending
  ON "OutboxJob" ("createdAt")
  WHERE "dispatchedAt" IS NULL;

```

WHY THE PARTIAL UNIQUE INDEX MATTERS

Two fans scanning the same code within the same millisecond is not hypothetical at an event. An application-level "have we already redeemed?" check loses that race every time; a unique index cannot. Catch the constraint violation and render "already used" — that is the correct implementation of §16's single-use rule.

04 Tenant scoping and authorization

§26 · every protected record

Two files, and no query anywhere in the codebase may skip them. `actor.ts` answers who is asking; `scope.ts` answers what they are allowed to see, as a Prisma `where` fragment you spread into every query.

```
// src/server/actor.ts
import { auth } from "@clerk/nextjs/server"
import { db } from "../db"

export type Actor = {
  userId: string; tenantId: string; roles: Role[]
  athleteId?: string; sponsorId?: string
}

export async function requireActor(): Promise<Actor> {
  const { userId: clerkId } = await auth()
  if (!clerkId) throw new UnauthorizedError()
  const u = await db.user.findUnique({ where: { clerkId } })
  if (!u) throw new UnauthorizedError()
  return { userId: u.id, tenantId: u.tenantId, roles: u.roles,
    athleteId: u.athleteId ?? undefined, sponsorId: u.sponsorId ?? undefined }
}
```

```
// src/server/scope.ts
import type { Prisma } from "@generated/prisma"

const DENY = { id: "__deny__" } // matches nothing; the safe default

export function athleteScope(a: Actor): Prisma.AthleteWhereInput {
  const t = { tenantId: a.tenantId }
  if (a.roles.some(r => ADMIN_ROLES.includes(r))) return t
  if (a.athleteId) return { ...t, id: a.athleteId }
  if (a.sponsorId) return {
    ...t,
    state_: "ACTIVE",
    orders: { some: { campaign: { sponsorId: a.sponsorId } } },
  }
  return { ...t, ...DENY }
}

export function orderScope(a: Actor): Prisma.CampaignOrderWhereInput {
  const t = { tenantId: a.tenantId }
  if (a.roles.some(r => ADMIN_ROLES.includes(r))) return t
  if (a.athleteId) return { ...t, athleteId: a.athleteId }
  if (a.sponsorId) return { ...t, campaign: { sponsorId: a.sponsorId } }
  return { ...t, ...DENY }
}
```

Field-level leaks are the failure mode here, not row-level ones. A sponsor legitimately sees an athlete row; they must not see `AthleteRate.amount` for a different sponsor's negotiated rate, and an athlete must never see `Campaign.budget`. Row scoping does not catch this. Use explicit `select` — never `include` — on every sponsor- and athlete-facing query, and add a lint rule that fails the build on a bare `findMany()` with no `select`.

05 Transactional job enqueue

The reason for Postgres queueing

An outbox row is written in the same transaction as the record that caused it. Either both land or neither does — so a campaign can never go live without its Zoho sync queued, and a queued sync can never point at a campaign that rolled back.

```

// src/server/outbox.ts
export async function enqueue(
  tx: Prisma.TransactionClient,
  tenantId: string, name: string, payload: unknown,
) {
  await tx.outboxJob.create({ data: { tenantId, name, payload: payload as Json } })
}

// src/server/domain/campaign.ts
export async function launchCampaign(actor: Actor, campaignId: string) {
  return db.$transaction(async (tx) => {
    const campaign = await tx.campaign.update({
      where: { id: campaignId, tenantId: actor.tenantId },
      data: { state: "ACTIVE" },
    })
    await tx.campaignOrder.updateMany({
      where: { campaignId, state: "ACCEPTED" }, data: { state: "ACTIVE" },
    })
    await enqueue(tx, actor.tenantId, "zoho.pushCampaign", { campaignId })
    await enqueue(tx, actor.tenantId, "notify.campaignLive", { campaignId })
    await audit(tx, actor, "campaign.launch", "Campaign", campaignId)
    return campaign
  })
}

```

WORKER/INDEX.TS – DRAIN TO PG-BOSS

```

const boss = new PgBoss({ connectionString: process.env.DATABASE_URL })
await boss.start()
await boss.work("zoho.pushCampaign", zohoPushCampaign)
await boss.work("notify.campaignLive", notifyCampaignLive)

setInterval(async () => {
  await db.$transaction(async (tx) => {
    const rows = await tx.$queryRaw<OutboxRow[]>`
      SELECT id, name, payload FROM "OutboxJob"
      WHERE "dispatchedAt" IS NULL
      ORDER BY "createdAt"
      LIMIT 100
      FOR UPDATE SKIP LOCKED`
    for (const r of rows) await boss.send(r.name, r.payload)
    if (rows.length)
      await tx.outboxJob.updateMany({
        where: { id: { in: rows.map(r => r.id) } },
        data: { dispatchedAt: new Date() },
      })
  })
}, 1000)

```

FOR UPDATE SKIP LOCKED

That clause is what lets you run two worker instances without either processing the same row. Drop it and a second worker — which you will add the first time a report generation blocks the queue — starts double-sending sponsor emails.

Four event types, four separate rows: **SCAN** when the QR resolves, **LANDING** when the page renders, **CLAIM** when the fan takes the offer, **REDEEM** when it is validated. §16 requires the separation and the funnel is worthless without it.

```
// src/app/t/[code]/route.ts – tracking redirect
import { after } from "next/server"

export async function GET(req: Request, { params }: { params: Promise<{ code: string }> }) {
  const { code } = await params
  const link = await db.trackingLink.findUnique({
    where: { code }, select: { destinationUrl: true, id: true, tenantId: true },
  })
  if (!link) return Response.redirect(FALLBACK_URL, 302)

  // log after the response is sent – the fan never waits on our write
  after(async () => {
    await db.rewardEvent.create({ data: {
      tenantId: link.tenantId, tokenId: link.id, type: "SCAN",
      city: req.headers.get("x-vercel-ip-city") ?? undefined,
      region: req.headers.get("x-vercel-ip-country-region") ?? undefined,
    }})
  })
  return Response.redirect(link.destinationUrl, 302)
}
```

```
// src/server/domain/reward.ts – redeem, race-safe
export async function redeem(token: string, fanEmail?: string) {
  const t = await db.rewardToken.findUnique({
    where: { token }, include: { reward: true },
  })
  if (!t || t.reward.state !== "ACTIVE") return { ok: false, reason: "invalid" } as const
  if (t.reward.expiresAt < new Date()) return { ok: false, reason: "expired" } as const

  try {
    await db.rewardEvent.create({ data: {
      tenantId: t.reward.tenantId, tokenId: t.id, type: "REDEEM", fanEmail,
    }})
  } catch (e) {
    if (isUniqueViolation(e)) return { ok: false, reason: "already_used" } as const
    throw e
  }
  if (fanEmail) await db.$transaction(tx =>
    enqueue(tx, t.reward.tenantId, "zoho.pushLead", { tokenId: t.id, fanEmail }))
  return { ok: true, offer: t.reward.offerText } as const
}
```

The redeem page must render without JavaScript. It is hit on a phone, on venue wifi, once. Server component, no client bundle, no font that blocks paint. Everything else in the product can be as heavy as it needs to be; this page cannot.

07 Zoho sync without the loop

Bi-directional sync fails in one predictable way: SponsorX writes to Zoho, Zoho's webhook fires back, SponsorX writes again. Two fields prevent it — the origin of the last write, and a hash of what was written.

```
// worker/jobs/zoho-push.ts
export async function zohoPushSponsor({ data }: Job<{ sponsorId: string }>) {
  const s = await db.sponsor.findUniqueOrThrow({ where: { id: data.sponsorId } })
  const payload = toZohoAccount(s)
  const hash = sha256(stableStringify(payload))
  if (s.lastSyncOrigin === "ZOH0" && s.lastSyncHash === hash) return // echo, drop it

  const zohoId = await zoho.upsertAccount({
    ...payload, External_Id: s.id, // dedupe key, per §18
  })
  await db.sponsor.update({ where: { id: s.id }, data: {
    zohoAccountId: zohoId, lastSyncOrigin: "SPONSORX", lastSyncHash: hash,
  }})
}
```

```
// src/app/api/v1/webhooks/zoho/route.ts
export async function POST(req: Request) {
  const raw = await req.text()
  if (!verifyZohoSignature(raw, req.headers.get("x-zoho-signature")))
    return new Response("bad signature", { status: 401 })

  const evt = ZohoWebhook.parse(JSON.parse(raw))
  const hash = sha256(stableStringify(evt.data))
  await db.$transaction(async (tx) => {
    await tx.sponsor.update({
      where: { zohoAccountId: evt.id },
      data: { ...fromZohoAccount(evt.data), lastSyncOrigin: "ZOH0", lastSyncHash: hash },
    })
    await enqueue(tx, evt.tenantId, "zoho.reconcile", { sponsorId: evt.id })
  })
  return Response.json({ ok: true })
}
```

08 Campaign Order acceptance

§12 · what makes it hold up

What matters legally is not a signature image — it is proof of exactly which text was shown, to whom, when, and from where. Hash the rendered body at display time and store the hash with the acceptance.

```

export async function acceptOrder(actor: Actor, orderId: string, req: RequestMeta) {
  const order = await db.campaignOrder.findFirstOrThrow({
    where: { id: orderId, ...orderScope(actor) },
  })
  if (order.state !== "SENT") throw new ConflictError("order not open for acceptance")
  if (actor.athleteId !== order.athleteId) throw new ForbiddenError()

  const agreement = await currentAgreement("CAMPAIGN_ORDER")

  return db.$transaction(async (tx) => {
    const acc = await tx.agreementAcceptance.create({ data: {
      tenantId: actor.tenantId, agreementId: agreement.id, userId: actor.userId,
      bodyHash: agreement.bodyHash, ip: req.ip, userAgent: req.userAgent,
      guardianId: await guardianFor(order.athleteId),
    }})
    await tx.campaignOrder.update({ where: { id: orderId }, data: {
      state: "ACCEPTED", acceptedAt: new Date(), acceptanceId: acc.id,
    }})
    await createDeliverablesFromJob(tx, order) // §13 step 7
    await enqueue(tx, actor.tenantId, "notify.orderAccepted", { orderId })
    await audit(tx, actor, "order.accept", "CampaignOrder", orderId)
  })
}

```

Do not build this until the templates exist. The Campaign Order and Content Collaboration Agreement are the documents missing from the operations binder. This code stores a hash of whatever text you give it — if that text has not been through counsel, you have built an audit trail for an unenforceable agreement.

09 The authorization matrix

§30 acceptance criterion

Not a checklist item at the end — a generated test that runs on every commit. Seed two tenants, two sponsors and two athletes, then assert every role against every resource.

```
// tests/authz.matrix.test.ts
const CASES = [
  // role,          resource,          owner,          expected
  ["SPONSOR_ADMIN", "campaign",          "own",          "allow"],
  ["SPONSOR_ADMIN", "campaign",          "other",         "deny" ],
  ["SPONSOR_ADMIN", "athleteRate.amount", "any",          "deny" ],
  ["ATHLETE",       "campaignOrder",     "own",          "allow"],
  ["ATHLETE",       "campaign.budget",    "any",          "deny" ],
  ["ATHLETE",       "athlete",           "other",         "deny" ],
  ["CAMPAIGN_MGR",  "campaign",          "own-tenant",   "allow"],
  ["CAMPAIGN_MGR",  "campaign",          "other-tenant", "deny"],
  ["GUARDIAN",      "campaignOrder",     "ward",         "allow"],
  ["GUARDIAN",      "earning",           "other",         "deny" ],
] as const

describe.each(CASES)("%s → %s (%s)", (role, resource, owner, expected) => {
  it(`is ${expected}ed`, async () => {
    const actor = await actorFor(role, owner)
    await expect(read(resource, actor)).resolves.toMatchAccess(expected)
  })
})
```

Add a row every time you add a resource. When someone widens a scope function to fix a bug, this suite is what tells them what else they just widened.

10 Environments and deploy

VARIABLE	USED BY	NOTE
DATABASE_URL	web + worker	Neon pooled for web, direct for worker and migrations.
DIRECT_URL	prisma migrate	Pooled connections break advisory locks during migration.
CLERK_SECRET_KEY	web	Plus the publishable key client-side.
R2_ACCOUNT_ID / R2_ACCESS_KEY_ID / R2_SECRET	web + worker	Bucket is private; all access via signed URLs.
ZOHO_CLIENT_ID / SECRET / REFRESH_TOKEN	worker	Worker only. The web app never calls Zoho directly.
ZOHO_WEBHOOK_SECRET	web	Signature verification on the inbound route.
TWILIO_* / RESEND_API_KEY	worker	All sends are jobs, so they retry.
SHORT_LINK_DOMAIN	web + worker	Own it. Do not use a third-party shortener.

Pipeline

1. **Migrations run as a release step, not in the build.** `prisma migrate deploy` against `DIRECT_URL`, before the new version takes traffic. A Vercel build that migrates will migrate once per preview deploy.
2. **Worker deploys before web** on any release that adds a job type, so a job enqueued by new web code has a handler waiting.
3. **Three environments:** preview branches share one Neon branch with seeded fake data; staging mirrors production and holds the pilot cohort during UAT; production has point-in-time restore on.
4. **Never seed production from the pilot spreadsheet by hand.** Write the importer as a job, run it against staging first, and keep it — you will run it more than once.

Vercel function limits are the one thing that will bite you. Report rendering, bulk imports and any Zoho backfill exceed the request timeout. They are worker jobs. If you find yourself raising a route's `maxDuration`, the work is in the wrong place.

11 Build order within a sprint

Every vertical slice lands in the same sequence. Deviating from it is how you end up with a UI that cannot be secured.

1. Zod contract in `src/contracts` — the schema is the specification.
2. Prisma model and migration.
3. Scope function, plus its rows in the authorization matrix.
4. Domain function in `src/server/domain`, with audit and outbox calls.
5. Route handler or server action — a thin wrapper that calls the domain function.
6. UI.
7. The E2E test that exercises the slice end to end.

Steps 1–4 are where the product lives; a reviewer should be able to read them without opening a component file. If a rule about who can see what exists only in a React component, it is not a rule — it is a suggestion the API will happily ignore.

Implementation guide for Phase 1 of the BTG SponsorX Master Development Blueprint v2.0. Section references (§) point to that document. Package versions verified against the npm registry in September 2026; re-check before Sprint 0 and pin whatever is current then.